VRS NLA Lookup API

Integration guide — calling from another Replit app · Generated 2026-05-29

OVERVIEW

Method: `GET`
Path: `/api/external/service-order/{SO}/nla`
Auth: `x-api-key header (machine-to-machine, no JWT)`
SO format: `DDDD-SSSSSSSS  (e.g. 8175-12345678)`

Returns every parts_nla submission for the service order, newest first, regardless of ticket status. followedNlaProcess is true when at least one exists.

SETUP

1. Add secrets to the other Replit app

Open the Secrets pane (lock icon) in the calling app and add:

VRS_LOOKUP_API_KEY: `vrs_sMk6AGRRubsQFRC9lSF-E71ST-4FKCoFkpSsByfwVoU`
VRS_API_BASE: `https://<your-vrs-app>.replit.app`

Replace <your-vrs-app> with this app's deployed URL (Deployments tab after publishing). Do not use the dev .repl.co preview URL.

2. Add the helper — server/vrsLookup.ts (Node 18+)

```
const VRS_BASE = process.env.VRS_API_BASE;
const VRS_KEY = process.env.VRS_LOOKUP_API_KEY;

export type VrsNlaSubmission = {
  id: number; serviceOrder: string; applianceType: string;
  requestType: string; ticketStatus: string;
  technician: { id: number; name: string; racId: string | null; phone: string | null };
  agent: { id: number; name: string } | null;
  escalatedBy: { id: number; name: string } | null;
  nlaResolution: string | null; nlaFoundPartNumber: string | null;
  partNumbers: string | null; technicianMessage: string | null; agentNotes: string | null;
  createdAt: string | null; claimedAt: string | null; statusChangedAt: string | null;
};
export type VrsNlaLookup = {
  serviceOrder: string; followedNlaProcess: boolean;
  nlaSubmissionCount: number; submissions: VrsNlaSubmission[];
};

export async function lookupNlaBySO(serviceOrder: string): Promise<VrsNlaLookup> {
  if (!VRS_BASE || !VRS_KEY) throw new Error("VRS_API_BASE and VRS_LOOKUP_API_KEY must be set");
  if (!/^\d{4}-\d{8}$/.test(serviceOrder)) throw new Error(`Bad SO format: ${serviceOrder}`);
  const url = `${VRS_BASE}/api/external/service-order/${serviceOrder}/nla`;
  const res = await fetch(url, { headers: { "x-api-key": VRS_KEY, accept: "application/json" } });
  if (!res.ok) throw new Error(`VRS lookup ${res.status}: ${await res.text()}`);
  return res.json() as Promise<VrsNlaLookup>;
}
```

3. Call it

```
import { lookupNlaBySO } from "./vrsLookup";

const result = await lookupNlaBySO("8175-12345678");
if (result.followedNlaProcess) {
  for (const s of result.submissions)
    console.log(`#${s.id} ${s.ticketStatus} tech=${s.technician.name} res=${s.nlaResolution ?? "none"}
`);
} else {
  console.log("No NLA submission found for this SO");
}
```

4. Shell sanity check

```
curl -H "x-api-key: vrs_sMk6AGRRubsQFRC9lSF-E71ST-4FKCoFkpSsByfwVoU" \
  https://<your-vrs-app>.replit.app/api/external/service-order/8175-12345678/nla
```

REFERENCE

Response shape

```
{
  "serviceOrder": "8175-12345678",
  "followedNlaProcess": true,
  "nlaSubmissionCount": 1,
  "submissions": [{
    "id": 49, "ticketStatus": "queued", "applianceType": "refrigeration",
    "technician": { "id": 10, "name": "...", "racId": "...", "phone": "..." },
    "agent": { "id": 7, "name": "..." } | null,
    "escalatedBy": { "id": 10, "name": "..." } | null,
    "nlaResolution": "part_found_vrs_ordered", "nlaFoundPartNumber": null,
    "partNumbers": "[\"WPW123456789\"]",
    "technicianMessage": "...", "agentNotes": "...",
    "createdAt": "...", "claimedAt": "...", "statusChangedAt": "..."
  }]
}
```

Status codes

- 200 — success; check nlaSubmissionCount
- 400 — bad SO format
- 401 — missing/invalid x-api-key
- 503 — VRS_LOOKUP_API_KEY not set on the server
- 500 — DB error

Rotating the key

- Update VRS_LOOKUP_API_KEY in Secrets on this VRS app.
- Update the same value in the calling app.
- No restart needed — value is read per request.

Common gotchas

- 401 - values don't match; check for trailing spaces.
- 503 - secret not set on the VRS app; set it and redeploy.
- 400 - SO must be 4 digits, hyphen, 8 digits.
- Network error - use the published deployment URL, not the dev preview.

Security note

The payload includes technician name / RAC ID / phone and free-text notes — wider than a yes/no check needs. Treat the key like a password: keep it server-side, rotate when a developer leaves, never embed it in client code. Ask if you want a stripped-down summary endpoint.